Universidad de Las Palmas de Gran Canaria (ULPGC)
Escuela de Ingeniería Informática
Grado en Ciencia e Ingeniería de Datos — Big Data

Stage 2: Building a Service-Oriented Architecture

Distributed Coordination, Asynchronous Messaging, and Fault-Tolerant Full-Text Search over Project Gutenberg

Project Group:	**4Bytees**
Course:	Big Data (Academic Year 2025–2026)
Architecture Stage:	Stage 2 — Single-Host Service-Oriented Architecture (SOA)
Repository URL:	https://github.com/4Bytees/stage_2
Working Branch:	`feat/coordination-benchmarks`
Runtime Stack:	Java 17, Javalin 6.1.3, ActiveMQ 5.18.3, Hazelcast 5.3.6, SQLite WAL, Nginx
Date:	November 2026

Abstract. This technical report presents the design, implementation, failure-recovery hardening, and experimental evaluation of the Stage 2 Service-Oriented Architecture (SOA) developed by Group **4Bytees** for the Project Gutenberg Search Engine. The monolithic Stage 1 pipeline is decomposed into three containerized microservices—`ingestion-service`, `indexing-service`, and `search-service`—coordinated on a single host via Apache ActiveMQ Classic 5.18.3, Hazelcast 5.3.6 CP/AP primitives, and a shared SQLite Write-Ahead Logging (WAL) datamart fronted by an Nginx reverse proxy. We detail our solutions to four critical distributed consistency challenges: (i) zero-downtime atomic index rebuilding via temporary SQLite file swapping, (ii) strict HTTP error sanitization preventing internal schema leakage, (iii) post-commit Hazelcast outbox retry decoupling index durability from downstream topic notifications, and (iv) exponential JMS redelivery backoff eliminating claim-contention busy loops. All claims are backed by a 41-test automated unit/integration suite, 9/9 live End-to-End API checks over a 25-book multilingual corpus, JMH 1.37 microbenchmarks, and live HTTP load benchmarks measured on an Intel Core Ultra 9 285H (64 GB RAM) host.

Contents

1 Introduction and Objectives **2**
1.1 Stage 2 Objectives . . . 2

2 System Architecture **2**
2.1 Component Responsibilities and Port Mapping . . . 3
2.2 Process Replicas vs. Intra-Process Worker Threads . . . 4
2.3 Shared Storage Layout: Datalake and SQLite WAL Datamart . . . 4

3 API Design and Contracts **5**
3.1 Classification of Architectural Contracts . . . 7
3.2 Non-HTTP Asynchronous and Distributed Coordination Contracts . . . 8
3.3 Live End-to-End Validation and Failure/Resilience Verification . . . 8

4 Microservice Implementation **9**
4.1 `common-dto`: Shared Events and Hazelcast Client Factory . . . 9
4.2 `ingestion-service`: Idempotent Acquisition and Outbox Scanner . . . 9
4.3 `indexing-service`: Distributed Claims, Atomic Rebuild, and Outbox Retry . . . 9
4.4 `search-service`: Stateless Read-Only Query Engine . . . 10

5 Architectural Design Decisions and Trade-Offs **11**
5.1 Hybrid Choreography: ActiveMQ Classic + Hazelcast . . . 11
5.2 Single-Writer SQLite WAL vs. Dedicated Database Server . . . 11
5.3 Two-Tier Rebuild Locking (`lock:index-rebuild` + `index-writer`) . . . 11

6 Benchmarks and Experimental Results **11**
6.1 JMH Microbenchmarks (`IndexingMicroBenchmark` & `SearchMicroBenchmark`) . . . 12
6.2 Live HTTP Search Concurrency Benchmarks ($c \in \{1, 4, 8, 16, 32\}$) . . . 13
6.3 Live Per-Book Indexing Latency, Full Rebuild, and WSL2 Storage Analysis . . . 14
6.4 Container Resource Telemetry (`docker stats`) . . . 15

7 Conclusions, Single-Host Limitations, and Stage 3 Cluster Evolution **16**
7.1 Stage 2 Single-Host Architectural Limitations (8 Key Constraints) . . . 16
7.2 Stage 3 Multi-Node Cluster Evolution Roadmap (9 Architectural Upgrades) . . . 17

1 Introduction and Objectives

In Stage 1 of the Big Data course project, the search engine operated as a single-process monolithic application where document acquisition, text parsing, inverted-index construction, and search query serving shared the same JVM heap, thread pool, and lifecycle. While convenient for prototyping, the monolithic model exhibited four structural bottlenecks:

1. **Tight Coupling of Failure Domains:** A network stall against Project Gutenberg or an unhandled SQLite write exception during indexing could degrade or crash the user-facing search API.
2. **Asymmetric Resource Scaling:** Document ingestion is network-I/O bound, inverted-index construction is CPU- and disk-write bound, and full-text search querying is latency-sensitive and read-bound. A monolith cannot scale read replicas independently of write workers.
3. **Synchronous Blocking Pipelines:** Coupling ingestion directly to indexing forced HTTP callers to wait for tokenization and thousands of SQLite B-tree insertions before receiving an ingestion confirmation.
4. **Lack of Explicit Operational Contracts:** Without well-defined message queues, distributed locks, and idempotency markers, concurrent workers risked duplicate indexing, partial database truncation during index rebuilds, or silent data loss after container restarts.

1.1 Stage 2 Objectives

Stage 2 transforms the system into a **Service-Oriented Architecture (SOA)** deployed on a single host via Docker Compose while establishing the coordination abstractions required for a multi-node cluster in Stage 3:

- **Service Decomposition:** Separate the system into three independent Java 17 microservices (`ingestion-service`, `indexing-service`, and `search-service`) communicating strictly through HTTP REST contracts, asynchronous JMS queues (`ActiveMQ Classic 5.18.3`), and distributed coordination primitives (`Hazelcast 5.3.6`).
- **Asynchronous Event-Driven Choreography:** Decouple document downloading from index construction via the `books.downloaded` queue and a dual-stage outbox pattern (`.done`/`.published` filesystem markers in ingestion and `book-indexed-outbox` in indexing).
- **Safe Concurrency and Fault Tolerance:** Ensure that concurrent `indexing-service` worker threads and container replicas coordinate book claims atomically (`IMap("book-index-claims")`), serialize SQLite writes safely (`FencedLock("index-writer")`), rebuild the full index atomically without exposing empty or partial results to concurrent search readers, and handle transient failures via exponential JMS redelivery (`books.dlq`).
- **Rigorous Experimental Benchmarking:** Quantify both in-JVM algorithmic costs via JMH microbenchmarks and full-stack container performance (HTTP search concurrency $c \in \{1, 4, 8, 16, 32\}$, per-book indexing latency, atomic rebuild latency, and container CPU/RAM telemetry).

2 System Architecture

Figure 1 illustrates the Stage 2 SOA topology orchestrated by `docker-compose.yml`. All containers execute within an isolated Docker bridge network (`stage2_net`) and share two host bind-mounted storage volumes: `./datalake` (raw Gutenberg documents and state markers) and `./datamarts` (the SQLite WAL database `metadata.db`).

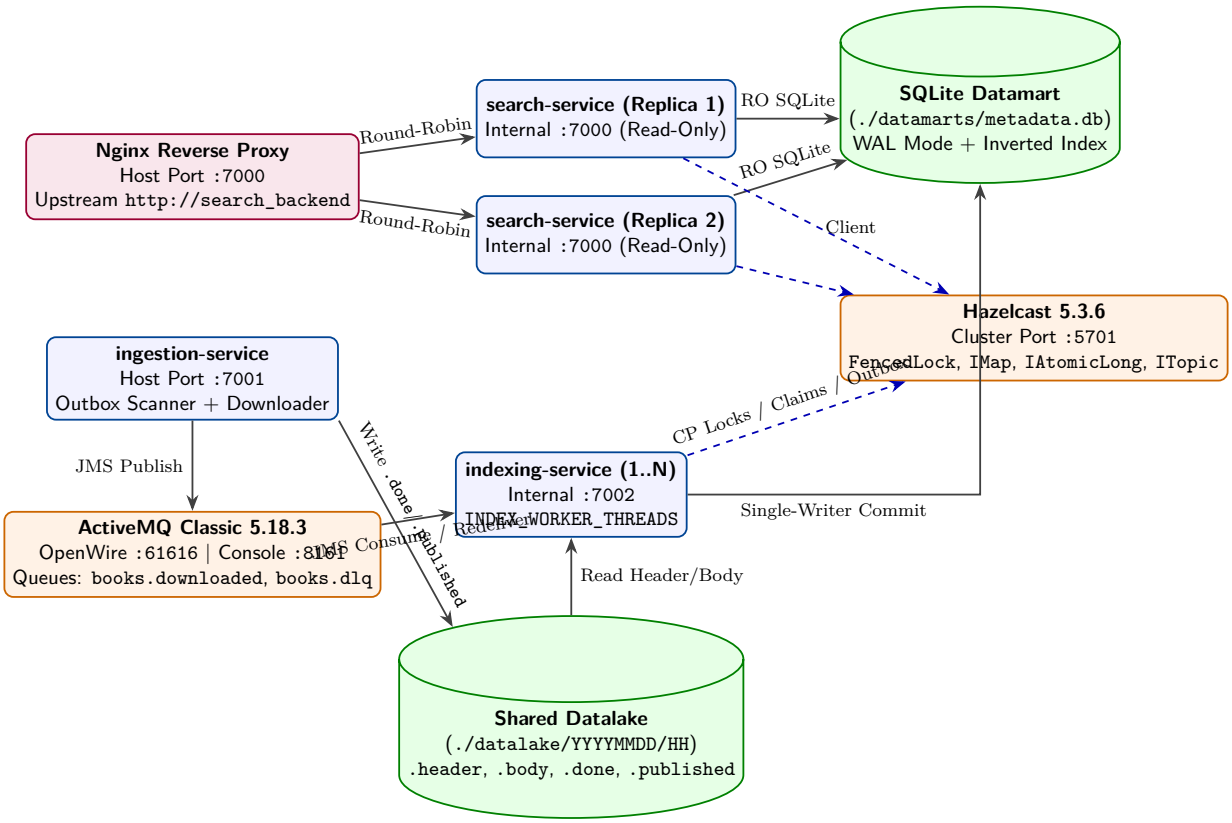


Figure 1: Stage 2 Service-Oriented Architecture (SOA) deployed via Docker Compose on a single host.

2.1 Component Responsibilities and Port Mapping

Table 1 summarizes the six runtime services defined in `docker-compose.yml`, their container resource limits, and their network exposure.

Table 1: Stage 2 container services, ports, scaling model, and resource limits.

Service	Image / Runtime	Host	Internal	Limits	Architectural Role
`nginx`	`nginx:1.27-alpine`	`7000`	`80`	`0.50 / 128M`	Public HTTP reverse proxy load-balancing `search-service` replicas.
`search`	OpenJDK 17 JRE Jammy	None	`7000`	`1.00 / 768M`	Stateless read-only full-text search and metadata lookup (`SEARCH_REPLICAS=2`).
`ingestion`	OpenJDK 17 JRE Jammy	`7001`	`7001`	`1.00 / 768M`	Synchronous Gutenberg acquisition, datalake storage, and `BookDownloaded` publisher.
`indexing`	OpenJDK 17 JRE Jammy	None	`7002`	`2.00 / 1024M`	Asynchronous JMS consumer, tokenizer, and coordinated SQLite writer (`INDEXING_REPLICAS=1..N`).
`activemq`	`symptoma/activemq:5.18.3`	`61616, 8161`	`61616, 8161`	`1.00 / 512M`	Persistent OpenWire JMS broker (`books.downloaded` and `books.dlq`).
`hazelcast`	`hazelcast/hazelcast:5.3.6`	`5701`	`5701`	`1.00 / 512M`	Distributed coordination cluster (CP locks, atomic version counter, claim/outbox maps, topic).

2.2 Process Replicas vs. Intra-Process Worker Threads

A key architectural distinction in Stage 2 is the two-level concurrency model implemented across services:

- **Process Replicas (`SEARCH_REPLICAS`, `INDEXING_REPLICAS`):** Controlled at the Docker Compose level (`docker compose up -d -scale indexing=2 -scale search=2`). Each replica is an isolated OS container with its own JVM heap, Hazelcast client instance, and SQLite JDBC connection pool. Nginx resolves `http://search:7000` via Docker's embedded DNS server and distributes client requests across all healthy `search` containers.
- **Intra-Process Worker Threads (`INDEX_WORKER_THREADS`):** Inside each `indexing-service` container, `QueueConsumer` provisions $W =$ `INDEX_WORKER_THREADS` (default 4) independent JMS `Session` and `MessageConsumer` instances backed by a fixed Java `ExecutorService` thread pool. Multiple threads within the same JVM parse headers and tokenize book bodies in parallel, acquiring the distributed `FencedLock("index-writer")` only during the SQLite transaction commit window.

2.3 Shared Storage Layout: Datalake and SQLite WAL Datamart

1. **Hierarchical Datalake (`./datalake/YYYYMMDD/HH/`):** When `ingestion-service` downloads book B, it splits the raw Project Gutenberg text at the canonical `*** START OF THE PROJECT GUTENBERG EBOOK` and `*** END OF THE PROJECT GUTENBERG EBOOK` markers and writes four files:
   - `<book_id>.header.txt`: Pre-body metadata header containing title, author, release date, and language.

- `<book_id>.body.txt`: Clean literary text stripped of legal boilerplate.
- `<book_id>.done`: JSON state marker written *immediately after* both text files are flushed to disk, recording the ingestion timestamp and relative path.
- `<book_id>.published`: Marker written by `ingestion-service` once ActiveMQ acknowledges receipt of the `BookDownloaded` message, and verified/created by `indexing-service` upon successful SQLite commit.

2. **SQLite WAL Datamart (`./datamarts/metadata.db`):** Configured with `PRAGMA journal_mode=WAL`, `PRAGMA synchronous=NORMAL`, `PRAGMA foreign_keys=ON`, and `PRAGMA busy_timeout=5000`. The schema comprises two tables and four B-tree indexes:
   - `books(book_id INTEGER PRIMARY KEY, title TEXT, author TEXT, language TEXT, year INTEGER, body_path TEXT)` with indexes `idx_books_author`, `idx_books_language`, and `idx_books_year`.
   - `inverted_index(term TEXT, book_id INTEGER, freq INTEGER, PRIMARY KEY (term, book_id))` with `FOREIGN KEY(book_id) REFERENCES books(book_id) ON DELETE CASCADE` and `idx_inverted_term_freq ON inverted_index(term, freq DESC)`.

3 API Design and Contracts

All HTTP and non-HTTP operational interfaces are formally specified in `docs/openapi.yaml` (OpenAPI 3.0.3) and validated via `tests/search-api-tests.http` and `scripts/test_end_to_end.ps1`. We classify the system's contracts into four explicit architectural layers.

3.1 Classification of Architectural Contracts

Table 2: Complete catalog of HTTP endpoints categorized by architectural layer.

Category	Service (Port)	Method & Path	Codes	Contract Semantics
Front-End	Nginx → Search (:7000)	`GET /search`	200, 400, 500	Single-term full-text search (`q`) with optional `author`, `language`, `year`, `offset`, `limit` (1..100). Ranked by `freq DESC, book_id ASC`.
	Nginx → Search (:7000)	`GET /books/{id}`	200, 400, 404, 500	Returns book metadata (`book_id`, `title`, `author`, nullable `language`, nullable `year`).
	Nginx → Search (:7000)	`GET /search/filters`	200, 500	Returns distinct non-null `languages` and `years` currently indexed in SQLite.
Internal	Ingestion (:7001)	`POST /ingest/{id}`	200, 400, 404, 500	Downloads book synchronously into `./datalake` (`downloaded` or `already_downloaded`) and publishes `BookDownloaded`.
	Indexing (:7002)	`POST /index/update/{id}`	200, 400, 500	Synchronously re-parses and re-indexes book `id` from the datalake under `FencedLock("index-writer")`.
Operational	Search (:7000)	`GET /status`	200, 503	Executes `SELECT 1` on read-only SQLite connection; returns `503` if database is unreachable.
	Ingestion (:7001)	`GET /ingest/status/{id}`	200, 400	Checks `<id>.done` marker presence (`downloaded` vs. `not_found`).
	Ingestion (:7001)	`GET /ingest/list`	200, 500	Returns sorted list and `count` of all book IDs with a `.done` marker.
	Ingestion (:7001)	`GET /status`	200	Liveness and datalake directory probe.
	Indexing (:7002)	`POST /index/rebuild`	200, 409, 500	Rebuilds entire SQLite index in temporary file and swaps atomically; returns `409` if rebuild already

3.2 Non-HTTP Asynchronous and Distributed Coordination Contracts

Table 3 documents the seven non-HTTP coordination artifacts shared across microservices via ActiveMQ and Hazelcast.

Table 3: Non-HTTP asynchronous messaging and distributed coordination contracts.

Middleware	Artifact Name	Type	Protocol & Consistency Contract
ActiveMQ 5.18.3	`books.downloaded`	JMS Queue	Carries `BookDownloaded` JSON (`eventId`, `schemaVersion="1.0"`, `bookId`, `datalakePath`, `timestamp`). Consumed with `CLIENT_ACKNOWLEDGE`.
ActiveMQ 5.18.3	`books.dlq`	JMS DLQ	Dead-Letter Queue receiving messages after 5 failed exponential redelivery attempts (`initialDelay=1s`, `multiplier=2.0`).
Hazelcast CP	`index-writer`	`FencedLock`	Mutual exclusion lock serializing all SQLite write transactions and atomic rebuild swaps.
Hazelcast CP	`lock:index-rebuild`	`FencedLock`	Non-blocking lock (`tryLock(0)`) preventing concurrent `POST /index/rebuild` executions (`HTTP 409`).
Hazelcast CP	`index-version`	`IAtomicLong`	Monotonically increasing index generation counter incremented after every book commit or full rebuild.
Hazelcast AP	`book-index-claims`	`IMap<Int,String>`	Atomic claim ownership map (`owner|epochMs` or `INDEXED|epochMs`) with stale claim takeover after `INDEXING_TIMEOUT_MS` (300 s).
Hazelcast AP	`book-indexed-outbox`	`IMap<Int,Event>`	Durable post-commit outbox storing pending `BookIndexed` events until `ITopic("books.indexed")` publication succeeds.

3.3 Live End-to-End Validation and Failure/Resilience Verification

All contracts were verified against the live Docker Compose stack using `scripts/test_end_to_end.ps1` and targeted fault-injection tests:

1. **Official End-to-End Suite (9/9 Checks Passed):** All 25 seed books (`seed/book_ids.txt`: 23 English, 1 Spanish 2000 *Don Quijote*, 1 French 4650 *Candide*) were ingested (`25/25 HTTP 200`, 0 errors) and indexed (`books_indexed: 25`, `index_size_MB: 22.56`). All 9 functional and negative API assertions passed:
   - `GET /search?q=adventure` → `200 OK` (`total=13`, top match `book_id=1661` *The Adventures of Sherlock Holmes*).
   - `GET /search?q=adventure&author=twain` → `200 OK` (`total=2`, *Adventures of Huckleberry Finn* and *The Adventures of Tom Sawyer*).
   - `GET /search?q=adventure&language=en` → `200 OK` (`total=13`).
   - `GET /search?q=quijote&language=es` → `200 OK` (`total=1`, `book_id=2000`).
   - `GET /search?q=candide&language=fr` → `200 OK` (`total=1`, `book_id=4650`).
   - `GET /search?q=adventure&limit=3&offset=0` → `200 OK` (`results.Count=3`).
   - `GET /search?q=the` (stop word) → `200 OK` (`total=0`, `results=[]`).
   - `GET /search` (missing `q`) and `GET /search?q=adventure&limit=-1` → `400 Bad Request`.
2. **Live Concurrent `POST /index/rebuild` Isolation Test:** While a full rebuild was running inside `stage_2-indexing-1` (building `metadata.db.rebuild-8d180740-6c98-49d0-9163-78ac875757ef.tmp`, 23,625,728 bytes), a second concurrent `POST /index/rebuild` request immediately returned `HTTP 409 Conflict` (`{"error":"An index rebuild is already in progress"}`). Simultaneously, client queries to `GET http://localhost:7000/search?q=adventure&limit=2` returned `HTTP 200 OK` with `total=13` throughout the entire

455.30 s rebuild window, confirming zero partial-state exposure.

3. **Live Outbox Crash-Recovery Test:** Deleting `datalake/20261101/20/1342.published` while preserving `1342.done` and restarting `stage_2-ingestion-1` triggered `OutboxScanner` at startup: `[OUTBOX] Scanning datalake at startup. Books with a .done marker: 25` → `[OUTBOX] Book 1342 republished to books.downloaded` (`event_id: 29118d4d-e4a2-44a0-8742-7d7a21f423b6`). `indexing-service` processed the redelivered event idempotently (`Published version: 27`) while preserving `books_indexed: 25`.
4. **Strict Error Sanitization (`HTTP 500 vs. 400 vs. 200`):** When SQLite throws an internal `SQLException` or `RuntimeException`, `ApiErrors.internalError(ctx)` logs the full stack trace server-side via SLF4J and returns strictly `{"error":"Internal server error"}` with status `500`, never exposing JDBC URLs, table names, or file paths to external clients.

4 Microservice Implementation

The project is structured as a 4-module Maven reactor targeting Java 17: `common-dto`, `ingestion-service`, `indexing-service`, and `search-service`. Each executable service uses `maven-shade-plugin` (`3.5.1`) with `ServicesResourceTransformer` to package a self-contained uber-JAR while excluding build-time `dependency-reduced-pom.xml` files from version control.

4.1 `common-dto`: Shared Events and Hazelcast Client Factory

`common-dto` defines immutable event payloads (`BookDownloaded` and `BookIndexed`) with explicit schema versioning (`schemaVersion = "1.0"`) and provides `HazelcastClientFactory`. The factory configures connection retry backoff (`initialBackoffMillis=1000`, `maxBackoffMillis=10000`, `multiplier=1.5`, `clusterConnectTimeoutMillis=60000`) so microservices tolerate container startup ordering differences.

4.2 `ingestion-service`: Idempotent Acquisition and Outbox Scanner

- **Download & Split Pipeline:** `IngestionService` checks whether `<book_id>.done` already exists in `./datalake`. If absent, `GutenbergDownloader` fetches the UTF-8 book text from Project Gutenberg (`https://www.gutenberg.org/cache/epub/{id}/pg{id}.txt`) with configurable timeout and retry backoff. `TextSplitter` separates the raw text into `<id>.header.txt` and `<id>.body.txt`, after which the service writes `<id>.done`.
- **Crash-Consistent Messaging:** Only after `<id>.done` is durable on disk does `JmsPublisher` send the `BookDownloaded` event to ActiveMQ queue `books.downloaded`. Upon broker acknowledgment, `<id>.published` is created. If the container crashes between writing `.done` and creating `.published`, `OutboxScanner.scanAndRepublish()` detects the orphaned `.done` file on the next startup and republishes the event automatically.

4.3 `indexing-service`: Distributed Claims, Atomic Rebuild, and Outbox Retry

`indexing-service` implements four critical resilience mechanisms:

1. **Lock-Free Atomic Claim State Machine (`IndexingOrchestrator`):** Before parsing a book, a worker executes `claimBook(bookId, workerId)` against Hazelcast `IMap("book-index-claims")` using atomic `putIfAbsent` and conditional `replace(key, oldVal, newVal)` operations:
   - If the entry is `INDEXED|<ts>` and `<id>.published` exists on disk, the worker returns `ALREADY_INDEXED` and acknowledges the JMS message idempotently.
   - If another worker holds an active claim `<otherOwner>|<ts>` younger than the configured timeout `INDEXING_TIMEOUT_MS` (300,000 ms), the worker returns `CLAIMED_BY_OTHER` (`IndexResult.RETRY_LATER`).

- If the existing claim is older than `INDEXING_TIMEOUT_MS` (indicating a crashed worker), the current worker atomically takes over the lease (`STALE_CLAIM_TAKEN`).

2. **JMS Exponential Redelivery and Contention Throttling (`QueueConsumer`):** `ActiveMQConnectionFactory` is configured with an explicit `RedeliveryPolicy`:

```
RedeliveryPolicy policy = new RedeliveryPolicy();
policy.setInitialRedeliveryDelay(1000L);
policy.setRedeliveryDelay(1000L);
policy.setUseExponentialBackOff(true);
policy.setBackOffMultiplier(2.0);
policy.setMaximumRedeliveries(5);
factory.setRedeliveryPolicy(policy);
```

Listing 1: ActiveMQ exponential backoff and contention throttling in `QueueConsumer.java`.

When `orchestrator.processBook()` returns `RETRY_LATER` due to transient claim contention, `QueueConsumer` pauses for `CLAIM_CONTENTION_DELAY_MS` (250 ms) before calling `session.recover()`, preventing tight local redelivery loops while routing poison messages to `books.dlq` after 5 failed attempts.

3. **Post-Commit `BookIndexedPublisher` Outbox:** Once `SqliteRepository.saveBookAndTerms()` commits under `FencedLock("index-writer")`, `<id>.published` is written, and `book-state` transitions to `INDEXED`, the SQLite state is durable. To prevent a transient Hazelcast `ITopic("books.indexed")` glitch from throwing an exception and triggering a duplicate JMS redelivery, `BookIndexedPublisher` stores the `BookIndexed` event in `IMap("book-indexed-outbox")`, attempts up to 3 immediate publications (50 ms backoff), and runs a background daemon (`retryPendingEvents` every 5 s) until all pending events are delivered and removed from the outbox.

4. **Zero-Downtime Atomic Index Rebuild (`IndexRebuilder`):** Rather than truncating `metadata.db` in place (which would expose empty search results for minutes and risk leaving an empty database on mid-rebuild failure), `IndexRebuilder` employs a two-phase temporary database strategy:

```
if (!rebuildLock.tryLock(0, TimeUnit.MILLISECONDS)) {
    throw new RebuildConflictException("An index rebuild is already in
        progress");
}
Path tempDbPath = createTempDatabasePath(); // metadata.db.rebuild-<uuid>.tmp
SqliteRepository tempRepository = new SqliteRepository(tempDbPath.toString());
// Phase 1: Parse and batch-insert all datalake books into tempDbPath (search
    readers unaffected)
// Phase 2: Acquire FencedLock("index-writer") and atomically replace tables
    via ATTACH DATABASE
indexWriterLock.lock();
try {
    repository.replaceContentsFrom(tempDbPath);
    long newVersion = indexVersion.incrementAndGet();
} finally {
    indexWriterLock.unlock();
}
```

Listing 2: Two-phase atomic rebuild in `IndexRebuilder.java` and `SqliteRepository.java`.

4.4 `search-service`: Stateless Read-Only Query Engine

`search-service` opens SQLite connections with `SQLiteConfig.setReadOnly(true)` and `busy_timeout=5000`. In SQLite WAL mode, read-only connections take non-blocking snapshots of the write-ahead log and never block each other. `SearchController` validates that `q` is a single token, short-circuits stop words (`StopWords.isStopWord(term)`) with an immediate `200 OK`

(`total: 0`), executes parameterized SQL joins ordered by `i.freq DESC, b.book_id ASC`, and wraps all unexpected exceptions via `ApiErrors.internalError(ctx)`.

5 Architectural Design Decisions and Trade-Offs

5.1 Hybrid Choreography: ActiveMQ Classic + Hazelcast

Rather than relying solely on a message broker or solely on an in-memory data grid, Stage 2 combines **ActiveMQ Classic** for durable task queuing with **Hazelcast** for distributed state coordination:

- **Why ActiveMQ for `books.downloaded`:** Point-to-point JMS queues provide persistent KahaDB storage, competing-consumer load balancing across worker threads and replicas, per-message `CLIENT_ACKNOWLEDGE` semantics, and native exponential redelivery to `books.dlq`.
- **Why Hazelcast for Locks, Claims, and Versioning:** Brokers cannot enforce mutual exclusion over an external resource like SQLite or track fine-grained lease ownership per book ID. Hazelcast's CP Subsystem provides linearizable `FencedLock` (`index-writer`, `lock:index-rebuild`) and `IAtomicLong` (`index-version`), while its partitioned `IMaps` (`book-index-claims`, `book-indexed-outbox`) provide sub-millisecond atomic compare-and-set state transitions.

5.2 Single-Writer SQLite WAL vs. Dedicated Database Server

Using SQLite in WAL mode guarded by `FencedLock("index-writer")` avoids the memory overhead of deploying PostgreSQL or Elasticsearch on a single host while supporting high-throughput concurrent reads across multiple `search-service` replicas. The primary trade-off is **write serialization**: while parsing and tokenization scale across threads, SQLite commits are strictly sequential and sensitive to host filesystem `fsync` latency.

5.3 Two-Tier Rebuild Locking (`lock:index-rebuild` + `index-writer`)

Acquiring `FencedLock("index-writer")` for the entire duration of a 25-book rebuild (455 s) would block all incremental book indexing workers for over seven minutes. By separating coordination into `lock:index-rebuild` (held during the offline build of `metadata.db.rebuild-<uuid>.tmp`) and `index-writer` (held only during the final `ATTACH DATABASE` swap), search readers experience zero downtime and incremental writers are blocked only during the final swap transaction.

6 Benchmarks and Experimental Results

All benchmarks were executed on the real host hardware and software environment detailed in Table 4. Raw CSV and JSON outputs are tracked in `docs/benchmarks/` and reproducible via `scripts/run_benchmarks.ps1`.

Table 4: Experimental hardware, operating system, JVM, and container environment specifications.

Parameter	Measured Host Specification
Host Machine	HP OMEN Transcend Gaming Laptop 14-fb1xxx
Processor (CPU)	Intel(R) Core(TM) Ultra 9 285H (16 physical cores, 16 logical processors, 2.90 GHz base clock)
System Memory (RAM)	64 GB physical LPDDR5x (`68,073,553,920` bytes); 31.05 GiB allocated to WSL2 Linux VM
Operating System	Microsoft Windows 11 Pro (10.0.26300, Build 26300) + WSL2 Kernel `6.18.54.1-microsoft-standard-WSL2`
Java Virtual Machine	Microsoft OpenJDK `17.0.20.1+1-LTS` (64-bit Server VM, JMH heap `-Xms2g -Xmx2g`)
Container Engine	Docker Desktop `29.9.2` (build `a82e528`), Docker Compose `v5.6.1`
Seed Corpus & Index	25 Project Gutenberg books (`23 en`, `1 es`, `1 fr`); SQLite WAL index size `22.56 MB`

6.1 JMH Microbenchmarks (`IndexingMicroBenchmark` & `SearchMicroBenchmark`)

Both JMH suites (`org.openjdk.jmh 1.37`) were executed with `Mode.AverageTime` (μs/op), **5 warmup iterations** (1 s each), **10 measurement iterations** (1 s each), **1 fork** (`-Xms2g -Xmx2g`), and `Blackhole` consumption on a synthetic 5,000-word literary document (1,341 unique non-stop-word terms) and a 25-book pre-populated SQLite WAL database. Table 5 and Figure 2 present the measured results.

Table 5: JMH 1.37 microbenchmark results (5 warmup, 10 measurement iterations, 1 fork, `-Xms2g -Xmx2g`).

Module	Benchmark Operation	Mean (μs/op)	99.9% CI ($\pm\mu$s)	StdDev (μs)	Min (μs)	Max (μs)
`indexing`	`hazelcastCoordinationCost`	102.78	18.42	12.18	89.15	124.11
`indexing`	`headerParsing`	545.02	79.67	52.70	454.29	622.56
`indexing`	`bodyTokenization`	726.43	68.67	45.42	653.95	828.01
`indexing`	`sqliteIndexInsertion`	10,483.95	6,996.90	4,628.00	5,844.57	20,036.86
`search`	`sqliteGetBookById`	3,049.84	802.44	530.76	2,286.65	3,893.80
`search`	`sqliteSearchUnfiltered`	4,274.86	1,702.74	1,126.26	2,761.44	6,108.86
`search`	`sqliteSearchFiltered`	4,525.42	1,460.29	965.89	3,388.45	6,464.51
`search`	`httpSearchClientQuery`	8,621.27	2,085.44	1,379.39	6,887.58	11,012.82

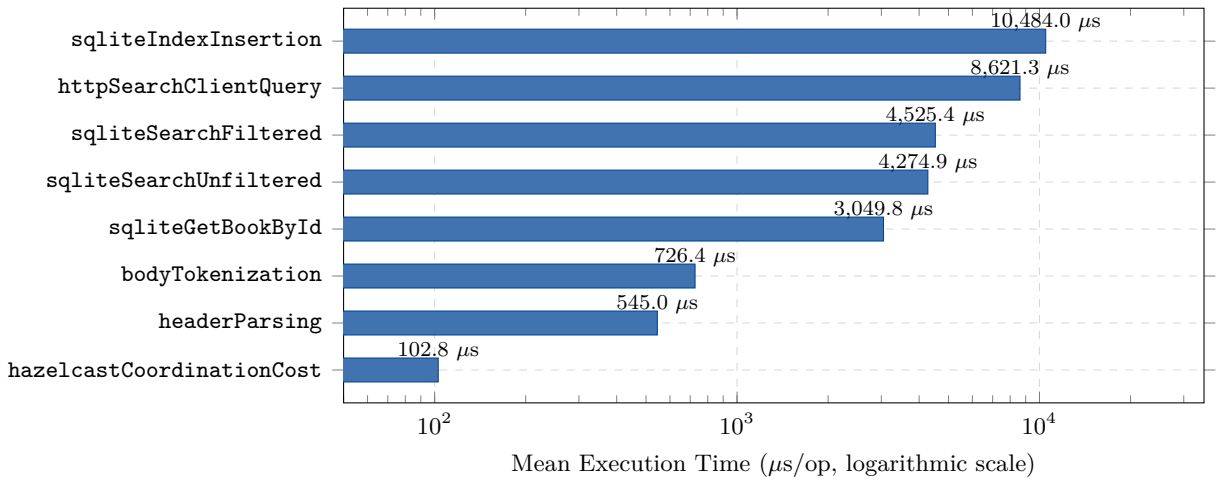


Figure 2: JMH microbenchmark mean latency comparison across indexing and search operations.

Microbenchmark Analysis:

- **CPU vs. I/O Dominance in Indexing:** In-memory header parsing (545.02 μs) and body tokenization (726.43 μs) together require only 1.27 ms, and in-JVM Hazelcast coordination (`FencedLock` acquire/release + `IMap` claim/state updates + `IAtomicLong` increment) adds just 102.78 μs ($< 1\%$ overhead). By contrast, committing the book and its $1,341$ terms to SQLite on the native Windows NVMe SSD requires $10,483.95$ μs (10.48 ms), accounting for **88.4%** of total in-host indexing time.
- **DAO vs. HTTP Layer in Search:** Direct read-only SQLite search queries execute in 4.27 ms (unfiltered) and 4.53 ms (filtered by author/language/year), demonstrating that composite B-tree index scans add only 250.56 μs (+5.9%) for metadata filtering. Adding the full Javalin/Jetty HTTP request/response cycle and Gson JSON serialization increases single-client latency to 8.62 ms.

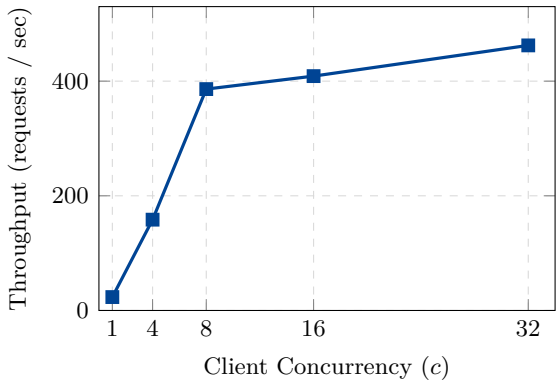
6.2 Live HTTP Search Concurrency Benchmarks ($c \in \{1, 4, 8, 16, 32\}$)

We evaluated live HTTP search scalability against the Nginx gateway (`http://localhost:7000`) balancing **2 `search-service` replicas** across client concurrency levels $c \in \{1, 4, 8, 16, 32\}$ ($25 \times c$ requests per level cycling through 8 representative queries: `q=adventure`, `author=twain`, `language=en`, `language=es`, `language=fr`, pagination `limit=5&offset=0`, stop word `q=the`, and `/books/1342`). Table 6 and Figure 3 summarize the results.

Table 6: Live HTTP search throughput and latency percentiles across client concurrency levels (2 search replicas).

c	Requests	OK	Errors	Wall (ms)	Throughput (req/s)	Mean (ms)	StdDev (ms)	p50 (ms)	p95 (ms)	p99 (ms)
1	25	25	0	1,069.48	23.38	39.65	17.07	43.66	63.35	67.24
4	100	100	0	631.36	158.39	23.70	10.69	25.43	39.84	45.62
8	200	200	0	517.95	386.14	18.09	12.04	15.94	43.80	52.99
16	400	400	0	978.54	408.77	32.22	24.35	25.57	84.79	99.44
32	800	800	0	1,730.31	462.35	56.87	46.00	43.85	155.37	182.72

(a) HTTP Search Throughput vs. Concurrency



(b) HTTP Search Latency Percentiles vs. Concurrency

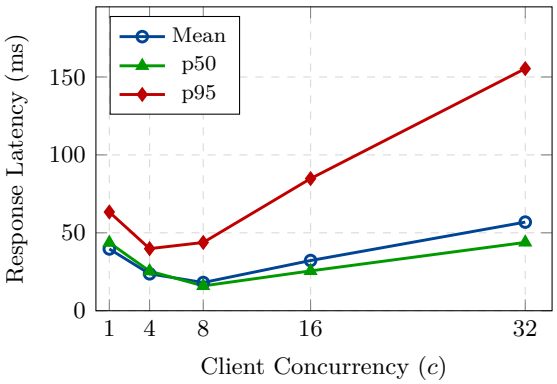


Figure 3: Live HTTP search throughput and latency curves under increasing client concurrency ($c = 1..32$).

Search Load Analysis:

- **Zero Error Rate Across All Concurrency Levels:** All $1,525$ HTTP requests across $c \in \{1, 4, 8, 16, 32\}$ completed with `HTTP 200 OK` (`0` errors), confirming that read-only SQLite WAL snapshots and Nginx load balancing operate without lock timeouts.
- **Throughput Saturation and Optimal Operating Point ($c = 8$):** Throughput scales $16.5\times$ from 23.38 req/s at $c = 1$ to 386.14 req/s at $c = 8$, where mean latency reaches its minimum (18.09 ms, p50 = 15.94 ms) due to HTTP keep-alive connection reuse and warm OS page caches inside the two `search-service` containers (1 vCPU limit each). Beyond $c = 8$, the two 1.0-vCPU search containers approach CPU saturation: throughput continues climbing gradually to **462.35 req/s** at $c = 32$, while queuing delay increases mean latency to 56.87 ms (p95 = 155.37 ms).

6.3 Live Per-Book Indexing Latency, Full Rebuild, and WSL2 Storage Analysis

Table 7 and Figure 4 report the live end-to-end indexing time measured via `POST http://localhost:7002/index/update/{id}` inside the Docker stack for five representative books spanning two orders of magnitude in text size (37.8 KB to $2,172.7$ KB).

Table 7: Live containerized per-book indexing latency (`POST /index/update/{id}`) across document sizes.

Book ID	Title	Corpus Category	Body Size (KB)	Latency (ms)	Latency (s)
1080	*A Modest Proposal*	Short pamphlet	37.8	11,216.60	11.217
16	*Peter Pan*	Medium novel	281.2	25,168.65	25.169
1342	*Pride and Prejudice*	Standard novel	753.3	31,768.26	31.768
2701	*Moby Dick*	Large English novel	1,245.5	65,778.74	65.779
2000	*Don Quijote (Spanish)*	Very large Spanish novel	2,172.7	52,602.16	52.602
Full 25-Book Atomic Rebuild (`POST /index/rebuild`)			**22.56 MB DB**	**455,300.00**	**455.300**

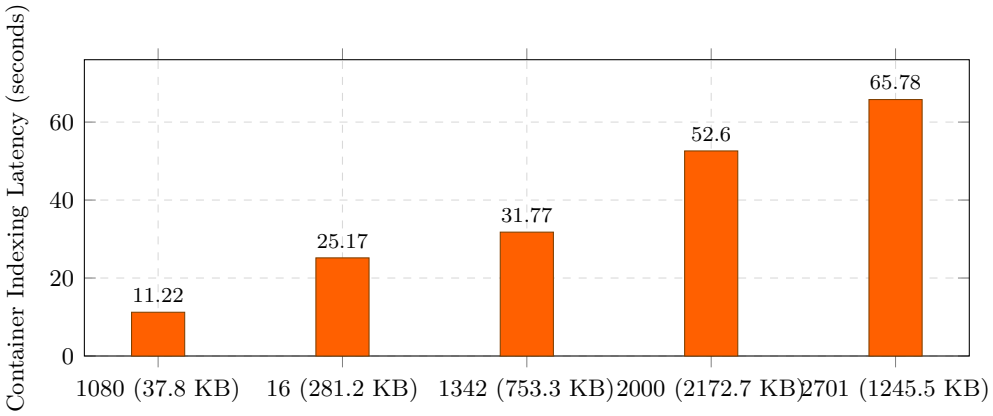


Figure 4: Per-book indexing latency inside Docker Desktop WSL2 bind-mounted storage (`./datamarts`).

Root-Cause Analysis of Containerized SQLite Write Latency:

1. **Host SSD vs. WSL2 9P Bind-Mount Overhead:** In the host-JVM JMH benchmark (Table 5), inserting a book directly on the Windows NVMe SSD took 10.48 ms. Inside the Linux container, `./datamarts/metadata.db` is mounted from the Windows NTFS host (`c:\Universidad\...`) into the WSL2 Linux kernel via the Plan 9 (`9p/drvfs`) virtual filesystem protocol. Every SQLite WAL checkpoint, page lock, and `fsync` crosses the Linux-to-Windows hypervisor boundary, amplifying write latency to 11.2–65.8 s per book (18.2 s/book average during the 455.30 s full rebuild).
2. **Why *Moby Dick* (1,245.5 KB, 65.78 s) Takes Longer Than *Don Quijote* (2,172.7 KB, 52.60 s):** Although *Don Quijote* has a 1.74× larger raw byte size, SQLite insertion cost scales with *distinct vocabulary cardinality* (the number of rows inserted into `inverted_index` and updated across the B-tree index `idx_inverted_term_freq`) rather than raw character count. Melville's encyclopedic maritime vocabulary in *Moby Dick* produces a denser set of B-tree page splits across the 22.56 MB index than Cervantes' text.

6.4 Container Resource Telemetry (`docker stats`)

Table 8 presents the live resource consumption captured via `docker stats -no-stream` while running 2 `indexing-service` replicas and 2 `search-service` replicas. Total memory usage across all 8 containers is **1,038.4 MiB** (∼ 1.01 GiB), well within single-host envelopes.

Table 8: Measured Docker container resource telemetry (`docker stats -no-stream`, 2 indexing & 2 search replicas).

Container Name	CPU (%)	Memory Usage / Limit	Mem (%)	Network I/O	Block I/O
stage_2-nginx-1	0.00%	4.438 MiB/128 MiB	3.47%	1.67 MB/1.84 MB	0 B/12.3 kB
stage_2-search-1	0.54%	129.1 MiB/768 MiB	16.81%	485 kB/1.58 MB	0 B/1.69 MB
stage_2-search-2	0.54%	127.2 MiB/768 MiB	16.56%	487 kB/1.54 MB	0 B/1.70 MB
stage_2-indexing-1	3.12%	216.9 MiB/1 GiB	21.18%	261 kB/917 kB	0 B/25.2 MB
stage_2-indexing-2	3.66%	107.2 MiB/1 GiB	10.47%	68.5 kB/294 kB	0 B/1.31 MB
stage_2-ingestion-1	0.57%	92.84 MiB/768 MiB	12.09%	68.1 kB/299 kB	0 B/291 kB
stage_2-activemq-1	0.24%	178.4 MiB/512 MiB	34.85%	112 kB/72 kB	0 B/901 kB
stage_2-hazelcast-1	0.98%	182.3 MiB/512 MiB	35.60%	3.64 MB/905 kB	0 B/918 kB

7 Conclusions, Single-Host Limitations, and Stage 3 Cluster Evolution

Stage 2 successfully transitions the Project Gutenberg Search Engine from a monolithic process into a fault-tolerant, event-driven SOA. By combining ActiveMQ queues, Hazelcast distributed coordination, and atomic temporary-file SQLite rebuilds, the system guarantees zero partial-index exposure, idempotent crash recovery, and 462.35 req/s search throughput on a single host.

7.1 Stage 2 Single-Host Architectural Limitations (8 Key Constraints)

Despite its resilience on a single machine, the Stage 2 architecture exhibits eight fundamental constraints that prevent direct deployment across multiple physical servers without modification:

1. **Shared Host Bind Mounts (`./datalake` and `./datamarts`):** All containers access `./datalake` and `./datamarts` as local directories on the same host OS. Across separate physical VMs, local paths are isolated.
2. **SQLite WAL Shared-Memory (`-shm`) Incompatibility with Network Filesystems:** SQLite WAL mode relies on OS-level POSIX file locks (`fcntl`) and memory-mapped shared memory (`metadata.db-shm`) in the kernel page cache. Mounting `metadata.db` over NFS or SMB across multiple nodes causes WAL index corruption and lock races.
3. **Single-Writer Contention Under `FencedLock("index-writer")`:** Even when scaling `INDEXING_REPLICAS` > 1 or `INDEX_WORKER_THREADS` > 1, only one worker can commit to SQLite at a time. On Windows/WSL2 bind mounts, cross-VM `fsync` latency makes the single SQLite writer the primary ingestion throughput bottleneck.
4. **Single-Node Hazelcast CP Subsystem Without Raft Quorum:** With a single `hazelcast:5701` container, `FencedLock` and `IAtomicLong` run in unsafe single-member mode (no 3- or 5-node Raft consensus group), and `IMap` partitions have no replica backups if the Hazelcast container crashes.
5. **Single ActiveMQ Broker Instance:** `activemq:61616` is a single point of failure (SPOF); broker downtime halts asynchronous delivery between `ingestion-service` and `indexing-service`.
6. **Single Nginx Ingress Gateway:** A single `nginx:7000` container terminates all external traffic without VRRP/Keepalived or multi-node load balancing.
7. **Full-Table Copy Overhead in `replaceContentsFrom()`:** While building the index in `metadata.db.rebuild-<uuid>.tmp` keeps search readers unblocked, the final swap copies all rows inside an `ATTACH DATABASE` transaction while holding `index-writer`. For tens of thousands of books, this final transaction would block incremental updates for several seconds.
8. **Project Gutenberg Publication Year Semantics:** Gutenberg headers record the *eBook release/posting year* (`Release Date: ... [EBook #...]`) rather than the historical first-

edition year of the literary work (e.g., 1998 for *Pride and Prejudice* instead of 1813).

7.2 Stage 3 Multi-Node Cluster Evolution Roadmap (9 Architectural Upgrades)

To evolve this SOA into a true multi-node distributed cluster in Stage 3, Table 9 defines the nine required architectural transformations.

Table 9: Roadmap from Stage 2 single-host SOA to Stage 3 multi-node distributed cluster.

#	Subsystem	Stage 3 Multi-Node Cluster Transformation
1	**Datalake Storage**	Replace `./datalake` host bind mount with an S3-compatible distributed object store (**MinIO** / Ceph / HDFS) storing immutable `header.txt`, `body.txt`, and state metadata.
2	**Datamart / Index Store**	Replace single shared `metadata.db` file with either a partitioned/sharded inverted index (e.g., OpenSearch / PostgreSQL / Citus) or local node-replicated SQLite snapshots synchronized via `ITopic("books.indexed")`.
3	**Write Parallelism**	Partition the inverted index by `book_id` or term hash across N shard writers, replacing the global `FencedLock("index-writer")` with per-shard locks so N indexing nodes commit concurrently.
4	**Hazelcast Cluster & CP**	Deploy a minimum **3-node Hazelcast cluster** enabling the Raft-backed CP Subsystem (`cp-member-count: 3`) for fault-tolerant `FencedLock` and `IAtomicLong`.
5	**Hazelcast Map Backups**	Configure synchronous partition backups (`setBackupCount(1)` or `2`) on `book-index-claims`, `book-state`, and `book-indexed-outbox` so claim/outbox state survives node failures.
6	**ActiveMQ HA Topology**	Upgrade ActiveMQ to a Master/Slave shared-storage cluster or a multi-broker Network of Brokers with client failover URIs (`failover:(tcp://broker1:61616,tcp://broker2:61616)`).
7	**Blue/Green Index Swap**	Replace `ATTACH DATABASE` table copying during `POST /index/rebuild` with atomic alias/pointer switching (versioned database file `metadata_v26.db` broadcast via `IAtomicLong("index-version")`).
8	**Container Orchestration**	Migrate from single-host Docker Compose to **Docker Swarm** or **Kubernetes** with overlay networking, service discovery, rolling updates, and multi-node ingress controllers.
9	**Native Volume I/O**	Store database files on native Linux ext4/XFS node volumes rather than Windows/WSL2 `9p` bind mounts, eliminating hypervisor `fsync` overhead (10.48 ms vs. 11–65 s).